ELSEVIER

Operations Research Letters 20 (1997) 75–79

operations research letters

A polynomial approximation scheme for problem $F2/r_j/C_{\max}$

Mikhail Y. Kovalyov[a,*], Frank Werner[b]

[a] *Institute of Engineering Cybernetics, Belarus Academy of Sciences, Suganova 6, 220012 Minsk, Belarus*
[b] *Otto-von-Guericke-Universität Magdeburg, Fakultät für Mathematik, PSF 4120, Magdeburg, Germany*

Received 1 January 1995; revised 1 August 1996

Abstract

We present a polynomial approximation scheme $\{H_\varepsilon\}$ for the strongly NP-hard problem of scheduling n jobs in a two-machine flow-shop subject to release dates. This scheme is based on a dynamic programming approach applied to a problem with rounded release dates and processing times. In comparison with Hall's polynomial approximation scheme, our scheme has a better time complexity estimation for small ε and sufficiently large n.

Keywords: Scheduling; Two-machine flow-shop; Polynomial approximation scheme; Dynamic programming

1. Introduction

We consider the following flow-shop scheduling problem. There are n independent non-preemptive jobs to be scheduled for processing on machines A and B. Each machine can handle at most one job at a time. Job j must be processed $a_j \geqslant 0$ time units on machine A and then $b_j \geqslant 0$ time units on machine B. The processing of job j cannot be started before a release date $r_j \geqslant 0$ and the processing of a job does not necessarily have to start on machine B immediately after its completion on machine A. The objective is to minimize the length of the schedule, i.e. the time at which the last job on machine B is completed. We denote the length of an arbitrary schedule S by $C(S)$ and the length of an optimal schedule by C^*.

According to the traditional notation for scheduling problems of Graham et al. [2], this problem is denoted by $F2/r_j/C_{\max}$. The problem is strongly NP-hard (see [10]), therefore there is no pseudopolynomial algorithm or fully polynomial approximation scheme for this problem unless $P = \mathrm{NP}$ [1].

In this paper, we develop a *polynomial approximation scheme* for the problem $F2/r_j/C_{\max}$, i.e. a family $\{H_\varepsilon\}$ of algorithms with the property that, for any $\varepsilon > 0$, H_ε produces a schedule of a length at most $(1+\varepsilon)C^*$ and it has a running time polynomial in n and exponential in $1/\varepsilon$.

As far as we know, there are only the following approximation results for the problem $F2/r_j/C_{\max}$. Potts [11] investigated the performance of five heuristics and showed that the best one guarantees a relative error $\varepsilon = \frac{2}{3}$. Hall [3] constructed a polynomial approximation scheme with $O((1+2/\varepsilon)^{(12+6\varepsilon)/\varepsilon^2}(1+(12+6\varepsilon)/\varepsilon^2)^{1+2/\varepsilon} n \log n)$ running time of each algorithm. This result is based on the notion of a $(1+\varepsilon)$-*approximation outline scheme* introduced by Hall and Shmoys [4–6] for solving single-machine scheduling problems. An outline scheme is a labeling of feasible solutions such that, for each feasible

* Corresponding author.

PII S0167-6377(96)00049-1

solution x, the associated label w, called an *outline*, provides concise information about x. More specifically, the length of w is bounded by a polynomial function of the length of x. For $\varepsilon > 0$, a $(1+\varepsilon)$-approximation outline scheme is an outline scheme and a polynomial-time algorithm R with the following property: given any outline w for an instance I of a problem, algorithm R delivers a feasible solution to I with a value at most $(1+\varepsilon)$ times the value of any feasible solution of I, which is labeled with w.

Our polynomial approximation scheme $\{H_\varepsilon\}$ for the problem $F2/r_j/C_{\max}$ is to apply a dynamic programming algorithm to a problem with rounded release dates and processing times. This approach is similar to the *rounding technique* described by Sahni [13] for knapsack-type problems. In comparison with Hall's scheme, our scheme has some benefits. Firstly, it is based on a well-known dynamic programming technique and, hence, an analysis of our scheme is rather simple. Secondly, it works more efficiently for small ε and sufficiently large n. Algorithm H_ε runs in $O(2^{8/\varepsilon+2}((n+1)/\varepsilon)^{6/\varepsilon+2})$ time. A similar approach is used by Kovalyov [8] to construct a polynomial approximation scheme for the problem of scheduling several two-machine flow-shops in parallel.

The rest of the paper is organized as follows. In the next section, we present some known results, which allow us to simplify an analysis of the problem and provide a basis for the application of a dynamic programming technique. In Section 3, we describe a dynamic programming algorithm for the problem with a fixed number of distinct release dates. This algorithm, applied to the problem with specifically rounded release dates and processing times, is our approximation algorithm H_ε. It is shown in the last section that the family of algorithms $\{H_\varepsilon\}$ forms a polynomial approximation scheme.

2. Preliminaries

Our dynamic programming approach is based on the results analogous to those used by Hall [3] for constructing a $(1+\varepsilon)$-outline scheme for the problem $F2/r_j/C_{\max}$. They are the following. We first observe that we may restrict our attention to so-called *permutation schedules*. They are schedules for which the job processing order is the same on machines A and B. Johnson [7] showed that there is always an optimal schedule that is a permutation one. If release dates of all jobs are equal, the problem can be solved by Johnson's algorithm [7]. It has a time complexity of $O(n \log n)$ and works as follows. First arrange the jobs with $a_j \leqslant b_j$ in the order of non-decreasing a_j, and then arrange the remaining jobs in the order of non-increasing b_j. The algorithm produces a permutation schedule. If this schedule is determined as a permutation $S = (i_1, \ldots, i_n)$, then its length can be found as

$$C(S) = \max_{1 \leqslant r \leqslant n} \left\{ \sum_{l=1}^{r} a_{i_l} + \sum_{l=r}^{n} b_{i_l} \right\}. \tag{1}$$

The following lemma allows us to focus on a restricted version of the problem $F2/r_j/C_{\max}$. This result is due to Hall [3] but we present the proof because it includes an algorithm, which is also used in our approximation scheme.

Lemma 1 (Hall [3]). *Given a polynomial approximation scheme for the restricted version of $F2/r_j/C_{\max}$ in which there is a fixed (but arbitrary) number of distinct release dates, then there is a polynomial approximation scheme for $F2/r_j/C_{\max}$.*

Proof. Define $r^* = \max_{1 \leqslant j \leqslant n}\{r_j\}$ and consider the following algorithm for the unrestricted version of the problem $F2/r_j/C_{\max}$. Round each release date r_j down to the nearest multiple of $\Delta = \varepsilon r^*/2$:

$$r'_j = \Delta \lfloor r_j/\Delta \rfloor, \quad j = 1, \ldots, n,$$

where $\lfloor x \rfloor$ is the largest integer no greater than x. Note that $\Delta \leqslant \varepsilon C^*/2$ since $r^* \leqslant C^*$. Clearly, the number of distinct release dates r'_j is at most $1 + r^*/\Delta \leqslant 1 + 2/\varepsilon$. Apply a $(1+\varepsilon/2)$-approximation algorithm for the problem with release dates r'_j. Add Δ to each job's starting time in the schedule just created to make it feasible for the original problem. Let R be the optimal objective value for the problem with release dates r'_j. Since the rounded problem is less constrained than the original one, we have $R \leqslant C^*$ and the length of the final schedule is bounded by

$$(1+\varepsilon/2)R + \Delta \leqslant (1+\varepsilon)C^*. \qquad \square$$

Let us assume that there are k distinct release dates denoted by $\rho_1 < \rho_2 < \cdots < \rho_k$. For notational convenience, we set $\rho_{k+1} = \infty$. For a given schedule S, we denote the starting time of job j on machine A by σ_j. Following Hall [3], we refer to the jobs j with $\rho_i \leqslant \sigma_j < \rho_{i+1}$ as the ith *interval set*. It is clear that only jobs with release dates $r_j \leqslant \rho_i$ can form the ith interval set.

The following lemma provides the basis for Hall's outline scheme and partially explains the choice of state variables in our dynamic programming approach.

Lemma 2 (Hall [3]). *For a given schedule S, rescheduling each interval set according to Johnson's algorithm and concatenating interval schedules delivers a schedule of length at most $C(S)$.*

Lemma 2 shows that the problem $F2/r_j/C_{\max}$ with k distinct release dates can be solved by constructing all k^n assignments of n jobs to k interval sets, scheduling each interval set according to Johnson's algorithm and concatenating interval schedules.

3. Dynamic programming

In this section, we present a dynamic programming algorithm for the problem $F2/r_j/C_{\max}$ with k distinct release dates assuming that all processing times are integers.

Suppose that the jobs are numbered according to Johnson's algorithm. Consider arbitrary schedules $S_1,\ldots,S_k$ in which jobs are sequenced from time zero in increasing order. Schedule S_i corresponds to ith interval set, $i = 1,\ldots,k$. Let us assign jobs to schedules $S_1,\ldots,S_k$ in all possible ways. Job j can be assigned to schedule S_i if $r_j \leqslant \rho_i$. Assume that schedule S_i consists of jobs $i_1 < i_2 < \cdots < i_p$ and job j is additionally assigned to this schedule. Then due to (1), the length of the new schedule can be calculated as follows:

$$C(S_i) = \max\left\{ \max_{1\leqslant r\leqslant p} \left\{ \sum_{l=1}^{r} a_{i_l} + \sum_{l=r}^{p} b_{i_l} \right\} + b_j, \sum_{l=1}^{p} a_{i_l} + a_j + b_j \right\}$$

$$= \max\{C(S_i) + b_j, A(S_i) + a_j + b_j\}, \qquad (2)$$

where $A(S_i) = \sum_{l=1}^{p} a_{i_l}$ is the total processing time on machine A.

We now present an algorithm to calculate the length $C(S)$ of a schedule S obtained by concatenating schedules $S_1,\ldots,S_k$. In this algorithm CON, schedules $S_1,\ldots,S_k$ are consecutively concatenated so that the current completion times T_A and T_B of machines A and B, respectively, are minimized. This strategy evidently leads to a schedule with minimal $C(S)$ value. The following parameters are assumed to be known for each schedule S_i: the total processing time on the first machine $A_i = A(S_i)$, the total processing time on the second machine $B_i = B(S_i) = \sum_{l\in\{S_i\}} b_l$ and the length $C_i = C(S_i)$. In iteration i of the algorithm, schedule S_i is concatenated with the current schedule. Denote by $T_A^{\text{old}}, T_B^{\text{old}}$ and $T_A^{\text{new}}, T_B^{\text{new}}$ the values of T_A and T_B calculated before and after concatenating S_i, respectively. Since T_A^{old} is the minimal current value of T_A and S_i corresponds to the ith interval set, jobs from S_i cannot be started before $E_i = \max\{\rho_i, T_A^{\text{old}}\}$. Therefore, $T_A^{\text{new}} = E_i + A_i$. Furthermore, since the sequence of jobs in S_i is fixed, $T_B^{\text{new}} = \max\{E_i + C_i, T_B^{\text{old}} + B_i\}$. We have $C(S) = T_B^{\text{new}}$, where T_B^{new} is calculated in iteration k. It is evident that algorithm CON runs in $O(k)$ time.

Let us denote by Σ the set of the schedules obtained by concatenating interval schedules $S_1,\ldots,S_k$ constructed in all possible ways. Lemma 2 shows that there is an optimal schedule corresponding to $\min\{C(S) \mid S \in \Sigma\}$.

Consider schedules $S, S' \in \Sigma$ obtained by concatenating $S_1,\ldots,S_k$ and $S'_1,\ldots,S'_k$, respectively. It follows from algorithm CON that $C(S) = C(S')$ if $C(S_i) = C(S'_i)$, $A(S_i) = A(S'_i)$ and $B(S_i) = B(S'_i)$, $i = 1,\ldots,k$. This observation and formula (2) provide the basis for the following dynamic programming algorithm H, which is similar to those used by Rothkopf [12] and Lawler and Moore [9] for parallel machine scheduling problems. The algorithm proceeds by computing a sequence of sets $X^{(0)}, X^{(1)},\ldots,X^{(n)}$. Each $X^{(j)}$ is a set of $3k$-tuples $(C_1,\ldots,C_k,A_1,\ldots,A_k,B_1,\ldots,B_k)$. Each tuple in $X^{(j)}$ corresponds to a certain assignment of the first j jobs into k interval schedules $S_1,\ldots,S_k$. The components C_i, A_i and B_i correspond to the length and the total processing time on machines A and B, respectively, for schedule S_i, $i = 1,\ldots,k$.

Algorithm *H*

Step 1 (Initialization): Set $X^{(0)} = \{(0,\dots,0)\}$.

Step 2 (Generation of $X^{(1)},\dots,X^{(n)}$): For $j = 1,\dots,n$, perform the following computations. Set $Y_i=\emptyset, i=1,\dots,k$. For each tuple $(C_1,\dots,C_k,A_1,\dots,A_k, B_1,\dots,B_k)\in X^{(j-1)}$ and $i=1,\dots,k$, do the following. If $r_j \leqslant \rho_i$, then set

$$Y_i = Y_i \cup (C_1,\dots,C_{i-1},\max\{C_i + b_j, A_i + a_j + b_j\},$$

$$C_{i+1},\dots,C_k,$$

$$A_1,\dots,A_{i-1},A_i + a_j,A_{i+1},\dots,A_k,$$

$$B_1,\dots,B_{i-1},B_i + b_j,B_{i+1},\dots,B_k).$$

Merge Y_i, $i = 1,\dots,k$, to obtain $X^{(j)}$. If we get the same tuples during the merge, store only one of them.

Step 3 (Determination of an optimal solution): For each tuple from $X^{(n)}$ apply algorithm CON to compute the length of the corresponding schedule. A tuple from $X^{(n)}$ with the minimal length corresponds to an optimal schedule, which is found by backtracking.

We now evaluate the time complexity of this algorithm. It is evident that the procedure of generating $X^{(j+1)}$ requires no more than $O(k|X^{(j)}|)$ time. Define $V = \max\{\sum_{i=1}^n a_i, \sum_{i=1}^n b_i\}$. It is not difficult to see that for each tuple $(C_1,\dots,C_k,A_1,\dots,A_k,B_1,\dots,B_k)\in X^{(j)}$, $j = 1,\dots,n$, we have $C_i \leqslant 2V$, $A_i \leqslant V$ and $B_i \leqslant V$, $i = 1,\dots,k$. Hence, the number of different tuples in $X^{(j)}$ is not greater than $(2V)^k V^{2k}$. This value can be reduced to $(2V)^k V^{2k-2}$ since only $k-1$ of the values $A_1,\dots,A_k$ and $k-1$ of the values $B_1,\dots,B_k$ are independent in each tuple. Thus, the time complexity of algorithm H can be estimated as $O(nk(2V)^k V^{2k-2})$, i.e. it is a pseudopolynomial algorithm if the number k of distinct release dates is a constant.

4. A polynomial approximation scheme

Let us consider the problem $F2/r_j/C_{\max}$ with k distinct release dates and arbitrary processing times. We define

$$\delta = \varepsilon V/(2n + 2)$$

and replace the given processing times a_j and b_j by scaled processing times

$$a'_j = \lfloor a_j/\delta \rfloor \quad \text{and} \quad b'_j = \lfloor b_j/\delta \rfloor.$$

Note that $V \leqslant C^*$ which implies $\delta(n+1) \leqslant (\varepsilon/2)C^*$. Suppose we have found an optimal schedule and its length R' for the problem with scaled processing times. Let C' denote the length of this schedule with respect to the original processing times and let R^* denote the length of an optimal schedule for the original problem with respect to the scaled processing times. Making use of the inequalities

$$\delta a'_j \leqslant a_j < \delta(a'_j + 1), \qquad \delta b'_j \leqslant b_j < \delta(b'_j + 1)$$

and noticing that, for any permutation schedule, if all a_j and b_j are increased by some value t, then the length of the schedule increases by at most $(n+1)t$, we obtain

$$C' < \delta R' + (n + 1)\delta \leqslant \delta R^* + (n + 1)\delta$$

$$\leqslant C^* + (n + 1)\delta \leqslant (1 + \varepsilon/2)C^*.$$

Thus, any exact algorithm for the problem with scaled processing times a'_j and b'_j is a $(1+\varepsilon/2)$-approximation algorithm for the problem with original processing times.

We now define a family of algorithms $\{H_\varepsilon\}$, where H_ε is the algorithm H applied to the problem with scaled release dates r'_j (see Section 2) and scaled processing times a'_j and b'_j. The value Δ (see Section 2) is added to each job's starting time in the final schedule to make it feasible for the original problem. Our main result is the following.

Theorem 1. *The family of algorithms $\{H_\varepsilon\}$ is a polynomial approximation scheme for $F2/r_j/C_{\max}$ with a time complexity of*

$$O(2^{8/\varepsilon+2}((n + 1)/\varepsilon)^{6/\varepsilon+2}).$$

Proof. Algorithm H applied to the problem with scaled processing times a'_j and b'_j and k distinct release dates is a $(1 + \varepsilon/2)$-approximation algorithm for the problem with original processing times. Let k be the number of distinct release dates r'_j. Then due to Lemma 1, algorithm H_ε delivers a schedule of length at most $(1 + \varepsilon)C^*$.

We now establish the time complexity of H_ε. We define $U = \max\{\sum_{i=1}^n a'_i, \sum_{i=1}^n b'_i\}$. Clearly, $U \leqslant V/\delta =$

$2(n+1)/\varepsilon$. Since $k \leqslant 1+2/\varepsilon$ (see proof of Lemma 1), the running time of algorithm H_ε is

$$O((n/\varepsilon)(2U)^{1+2/\varepsilon}U^{4/\varepsilon}),$$

from which the required time complexity is derived. □

For an arbitrary instance of the problem $F2/r_j/C_{\max}$, we cannot say whether our scheme or Hall's [3] scheme is better from the computational point of view. However, a simple estimation of the time complexities shows that our scheme can be expected to be faster for instance for $\varepsilon \leqslant \frac{1}{3}$ and $n \leqslant 20\,000$, $\varepsilon \leqslant \frac{5}{12}$ and $n \leqslant 1000$, $\varepsilon \leqslant 1/2$ and $n \leqslant 500$ or $\varepsilon < \frac{2}{3}$ and $n \leqslant 100$. If $\varepsilon \geqslant \frac{2}{3}$, then Potts' [11] heuristic is preferable, since it guarantees $\varepsilon = \frac{2}{3}$ and works more efficiently. It should be noted that the space requirement of our scheme is the same as its time requirement while Hall's scheme and Potts' heuristic both require a linear space.

Acknowledgements

The authors have been supported by Deutsche Forschungsgemeinschaft (Project ScheMA) and by INTAS (Project INTAS-93-257).

References

[1] M.R. Garey and D.S. Johnson, *Computers and Intractability: A Guide to the Theory of* NP-*Completeness*, W.H. Freeman and Co., New York, 1979.

[2] R.L. Graham, E.L. Lawler, J.K. Lenstra and A.H.G. Rinnooy Kan, "Optimization and approximation in deterministic sequencing and scheduling: a survey", *Ann. Discrete Math.* **5**, 287–326 (1979).

[3] L.A. Hall, "A polynomial approximation scheme for a constrained flow-shop scheduling problem", *Math. Oper. Res.* **19**, 68–85 (1994).

[4] L.A. Hall and D.B. Shmoys, "Approximation algorithms for constrained scheduling problems", *Proc. IEEE 30th Ann. Symp. Foundations of Computer Science*, 1989.

[5] L.A. Hall and D.B. Shmoys, "Near-optimal sequencing with precedence constraints", *Proc. Math. Prog. Soc. Conference on Integer Programming and Combinatorial Optimization*, University of Waterloo, 1990.

[6] L.A. Hall and D.B. Shmoys, "Jackson's rule for single-machine scheduling: making a good heuristic better", *Math. Oper. Res.* **17**, 22–35 (1992).

[7] S.M. Johnson, "Optimal two- and three-stage production scheduling with setup times included", *Naval Res. Logist. Quart.* **1**, 61–68 (1954).

[8] M.Y. Kovalyov, "Scheduling two-machine flow-shops in parallel", preprint 3, Institute of Engineering Cybernetics, Belarus Academy of Sciences, Minsk, 1993.

[9] E.L. Lawler and J.M. Moore, "A functional equation and its application to resource allocation and sequencing problems", *Management Sci.* **16**, 77–84 (1969).

[10] J.K. Lenstra, A.H.G. Rinnooy Kan, and P. Brucker, "Complexity of machine scheduling problems", *Ann. Discrete Math.* **1**, 343–362 (1977).

[11] C.N. Potts, "Analysis of heuristics for two-machine flow-shop sequencing subject to release dates", *Math. Oper. Res.* **10**, 576–584 (1985).

[12] M.H. Rothkopf, "Scheduling independent tasks on parallel processors", *Management Sci.* **12**, 437–447 (1966).

[13] S. Sahni, "General techniques for combinatorial approximation", *Oper. Res.* **25**, 920–936 (1977).